Practical Problems: Loops

Learning Outcome Addressed

- Recognize, interpret, and write programs using while loops and for loops.
- Recognize, interpret, and write programs that iterate through lists and strings with for loops.
- Evaluate provided test sets and write new test sets to verify that code works as expected.

This assignment allows you to apply what you have learned in this module about while and for loops, using lists and strings with for loops, and assessing and writing test sets. Follow the instructions below to complete problems one to twelve. If you get stuck on a problem without making any progress, please reach out to the learning facilitator for assistance.

1. [10 pts] n_ones

Write the function **n_ones(n)** that takes in a non-negative integer n and returns the integer consisting of exactly n 1's.

So n_ones(0) == 0, n_ones(1) == 1, n_ones(2) == 11, n_ones(3) == 111, and so on.

You may not use strings in this problem.

2. [10 pts] smallest_factor

We say that m is a factor of n if m divides n without a remainder. Write the function **smallest_factor(n)** that takes in a positive integer n and returns the smallest factor of n larger than 1. If n is 1, then you should return 1.

3. [10 pts] largest_factor

Write the function **largest_factor(n)** that takes in a positive integer n and returns the largest factor of n less than n. If n is 1, then you should return 1.

Do not call the "smallest_factor" function to solve this.

4. [10 pts] num_odd_digits

Write the function **num_odd_digits(n)** that takes in an integer n and returns the number of odd digits in the number.

5. [10 pts] longest_string_length

Write the function **longest_string_length(L)** that takes a list of strings (e.g. ["hi", "hello", “howdy”]) and returns the length of the longest string in the list.

6. [10 pts] average_of_primes

Write the function **average_of_primes(L)** that takes in a list of integers and returns the average of all the prime numbers in the list.

For convenience, the is_prime function has been implemented for you as it is written in the lecture video. If you’d like an extra challenge, try writing it from scratch!

7. [10 pts] is_digit

Write the function **is_digit(s)** that takes in a string (e.g. “cs110”) and returns True if there is at least one character in the string, and all the characters in the string are digits. The function returns False otherwise.

Hint: If c denotes a character, then we can check if c is a digit as follows: “0” <= c <= “9”

8. [10 pts] is_lower

Write the function **is_lower(s)** that takes in a string and returns True if there is at least one letter in the string and every letter in the string is lowercase. The function returns False otherwise.

For example, the input “Hello!” should return False, but input “hello!” should return True since all letters are lowercase. We don’t care about special character.

Hint: If c denotes a character, then we can check if c is a lowercase letter as follows: “a” <= c <= “z”
Similarly, we can check if c is an uppercase letter as follows: “A” <= c <= "Z".

9. [10 pts] is_alpha

Write the function **is_alpha(s)** that takes in a string and returns True if every character in the string is a letter (upper or lowercase) and there is at least one character in the string. The function returns False otherwise.

10. [10 pts] is_alpha_num

Write the function **is_alpha_num(s)** that takes in a string and returns True if there is at least one character and every character in the string is a letter or a digit. The function returns False otherwise.

Your function should call the is_alpha and is_digit functions that you have already written.

11. [25 pts] collatz

The Collatz Conjecture is a very famous open problem in mathematics. It states the following.

Start with any positive number n. If n is even, divide it by 2, and if it is odd, multiply it by 3 and add 1. Repeat this process with the resulting number. Eventually, you will end up with the number 1.

Mathematicians are nowhere near proving this statement. If you would like to learn more about it, watch this Youtube video.

Your task is to write a function **collatz(n)** that takes a positive integer n as input, and returns the number of steps it takes to reach the number 1 when we apply the rules described above.

12. [25 pts] longest_bit_run

Write the function **longest_bit_run(s)** that takes in a string consisting of only 0’s and 1’s. The returned value is the length of the longest run of 0’s or 1’s in the string.

For example, if the string is s = "0011101111000", then the output should be 4 since there is a run of 4 1’s, and that is the longest run in the string.